# Course Project Guidelines: Macroscopic System Modeling & Complexity Management

**Course:** Models and Algorithms in Networks and Systems (MAiNS)

**Instructor:** Prof. Weiqiang Sun

**Contact:** sunwq@sjtu.edu.cn, http://weiqiangsun.me/ | http://health.sjtu.edu.cn/home/weiqiangsun/

**Deliverable Type:** Group / Individual Project (3 Students max per Group)

**Focus:** Macroscopic System Modeling, System Architecture, and Abstraction

---

## 1. Project Overview & Objectives

In the second half of MAiNS, our focus shifts from **microscopic abstraction** (queueing dynamics, random access protocols, and probabilistic behavior) to **macroscopic complexity management**.

Large-scale Networks and Systems, agentic planning frameworks, and automated infrastructure design engines fail not because the underlying mathematics are wrong, but because **system complexity explodes**. As component counts grow, state space expands non-linearly, interactions become non-deterministic, and domain rules become intricate.

The objective of this project is to apply system architecture principles, state management techniques, and domain modeling abstractions to engineer or reverse-engineer a complex software system.

---

## 2. Project Options

You may select **one** of the following four project tracks:

### Option 1: Reverse Engineer the Agentic Network Planner (ANP)

- **Context:** ANP is a domain software designed for LLM-assisted optical access network planning
- **Goal:** From the exposed software functions, formalize its architectural blueprint—its structure, data flow, state management, etc.
- **Primary Complexity Challenge:** *Behavioral & Orchestration Complexity* (role-based access control, multi-stage handover, versioning).
- **Website:** ANP on AWS
- **Note:** Do not overload or hack the system - it is in production.

### Option 2: AI Infrastructure Planner (AI-Infra)

- **Context:** Modern AI data centers require multi-layered, multi-domain knowledge in planning and design—from land to building, from GPUs to networked clusters. This project aims to build a minimal viable planning and design software for modern AIDCs.
- **Goal:** Architect on and extend an ongoing open-source project.

- **Primary Complexity Challenge:** *Domain & Structural Complexity* (multi-layered abstraction, strict physical/logical constraints, high-dimensional search spaces).
- **Code Repo:** AI-INFRA

### Option 3: Ontology Workbench — OntoForge

- **Context:** Unstructured research literature contains deep domain insights that are difficult to organize systematically.
- **Goal:** Design an LLM-backed pipeline that ingests domain literature, extracts semantic entities and relationships into a formal graph/ontology, and provides automated mechanisms to identify research gaps.
- **Primary Complexity Challenge:** *Semantic & Knowledge Complexity* (formal schema definition, entity resolution, graph-theoretic abstractions).
- **Code Repo:** OntoForge

### Option 4: Self-Defined Project

- **Context:** Open to specialized research topics or custom system ideas.
- **Goal:** Propose, architect, and implement a complex networked or agentic system.
- **Requirement:** Must be pre-approved by the instructor via a 1-page proposal demonstrating that the system contains non-trivial structural, behavioral, or semantic complexity.

---

## 3. The Complexity Management Framework

Regardless of your chosen track, your system architecture and project report **must explicitly address three core dimensions of complexity**:

| Dimension | Description & Key Architectural Abstractions |
|---|---|
| **1. Structural Complexity** | Layered Architecture, Module Boundaries, State/Logic Separation, Clean API Interfaces |
| **2. Behavioral & Orchestration** | State Machines, Event-Driven Communication, Asynchronous Control, Versioning, Stage Handover, Context Sharing |
| **3. Semantic & Domain** | Formal Ontologies, Schemas, Graph Topologies, Constraint Systems, Data Registries |

---

## 4. Required Deliverables

Your final project submission must consist of two components:

1. **Working Codebase / Implementation:** Hosted on a Git repository (GitHub/GitLab) with clear build instructions, clean modularity, and automated unit/integration tests.
2. **System Architecture & Complexity Report:** A formal engineering report (MD/PDF, 8–12 pages) structured according to the submission template provided below.

---

## 5. Standardized Submission Template

*Use the following Markdown structure for your written report.*

```
# [Project Title]
**Course:** MAiNS (Models and Algorithms in Networks and Systems)
**Project Track:** [Option 1 / Option 2 / Option 3 / Option 4]
**Team Members:** [Name 1, Name 2, Name 3]
**Repository Link:** [URL]

---

## Executive Summary
A concise (250-word) summary of the system, its core domain, the primary
complexity challenges faced, and your architectural solution.

---

## 1. System Domain & Problem Statement
* Describe the macroscopic problem your system solves.
* Detail the operational scale, inputs, outputs, and primary constraints.
* Highlight why a naive/monolithic approach fails due to complexity
explosion.

---

## 2. Macroscopic Architectural Specification
Provide clean, professional visual diagrams for each sub-section (e.g.,
Mermaid, C4 diagrams, or Graphviz):

### 2.1 High-Level Component Architecture
* Diagram the system containers, core modules, and external boundaries.
* Define clear separation of concerns (e.g., Control Plane vs. Data Plane,
Layered Abstractions, Orchestration vs. Execution).

### 2.2 Data Models & Schema Design
* Formalize data structures at rest and in transit.
* Show ER diagrams, class/ontology hierarchies, or data registries.

### 2.3 Behavioral State Machine & Control Flow
* Provide sequence diagrams or formal Finite State Machines (FSM) showing
how the system transitions across execution steps.
* Explain asynchronous execution, event buses, multi-stage handover,
versioning, or role-based access control (RBAC).

---

## 3. Complexity Management Evaluation
Explicitly demonstrate how your design addresses the three MAiNS complexity
dimensions:

| Complexity Dimension | Identified Bottleneck / Anti-Pattern | Applied
Architectural Abstraction | Quantitative / Qualitative Outcome |
| :--- | :--- | :--- | :--- |
| **Structural** | *e.g., Tightly coupled modules across layers* | *e.g.,
```

```
*Strict Layered Architecture & API boundaries* | *Reduced cross-layer
dependencies* |
| **Behavioral** | *e.g., Context loss during multi-stage planning* |
*e.g., Explicit Stage Handover & Context Sharing protocols* | *Zero loss of
state across agent transitions* |
| **Semantic** | *e.g., Unstructured LLM outputs* | *e.g., Formal JSON
Schema + Ontology Registry* | *100% parseable structured graph* |

---

## 4. Implementation & Execution
* Detail key software engineering choices (e.g., languages, design
patterns, frameworks).
* Explain how the system enforces Role-Based Access Control (RBAC), data
registry safety, or state immutability if applicable.
* Show execution logs, API response traces, or CLI outputs demonstrating
successful operation.

---

## 5. Verification, Scaling & Trade-off Analysis
* **Scaling Limits:** Analyze where the architecture will experience
performance degradation as component or agent counts grow (O(N), O(N^2),
memory footprints, latency).
* **Trade-off Matrix:** Evaluate decisions made (e.g., Strong Consistency
vs. Eventual Consistency, In-Memory Registry vs. Database Persistence).

---

## 6. Conclusion & Future Work
* Summary of lessons learned in macroscopic system design.
* Remaining architectural bottlenecks and proposed future refactoring.

---

## References & Appendix
* Academic papers, open-source documentation, and external libraries used.
```

---

## 6. Evaluation Criteria & Grading Rubric

Projects will be evaluated out of **100 base points**, with an additional **15 bonus points** available for live software demonstrations:

| Criteria | Points | Focus |
|---|---|---|
| **Macroscopic Architecture & Diagrams** | 25 pts | Rigor, completeness, and clarity of system diagrams (C4 model, state machines, component decoupling, layered design). |

| Criteria | Points | Focus |
| --- | --- | --- |
| **Complexity Management Rigor** | 25 pts | How effectively the design mitigates structural, behavioral (RBAC, versioning, stage handover, context sharing), and semantic complexity explosion. |
| **Domain Precision & Realism** | 25 pts | Accuracy in representing domain constraints (e.g., optical access networks, AIDC physical/logical layers, formal ontologies). |
| **Analysis & Trade-off Evaluation** | 15 pts | Depth of the scaling analysis, failure mode identification, and architectural self-assessment. |
| **Code Quality & Modular Design** | 10 pts | Code organization, separation of concerns, documentation, test coverage, and build cleanliness. |
| **BONUS: Real-Life Software Demo** | **+15 pts** | Live presentation or screencast demonstrating a fully functional, end-to-end execution of the software handling real-world scenarios or dynamic user inputs. |